IT UNIVERSITY OF COPENHAGEN

Machine Learning Project

Group Clustered Together (k=3)

December 2025

Emil Sander Korczak

ekor@itu.dk

Joakim Smidsgaard Andersen

smja@itu.dk

Stella Petrova Boneva

stpb@itu.dk

BSMALEA1KU

1 Introduction

Predicting claim risk is a central challenge in insurance since it directly drives pricing and how much risk an insurer can take on. Underestimated risk may lead to major financial losses, while an overestimated risk may lead to a loss of customers[1]. Therefore, good risk prediction is crucial for insurers to stay financially stable while offering reasonable and competitive premiums. In this project, we will explore different machine learning methods to predict claim rate and model insurance risk.

2 Data Analysis

2.1 Dataset

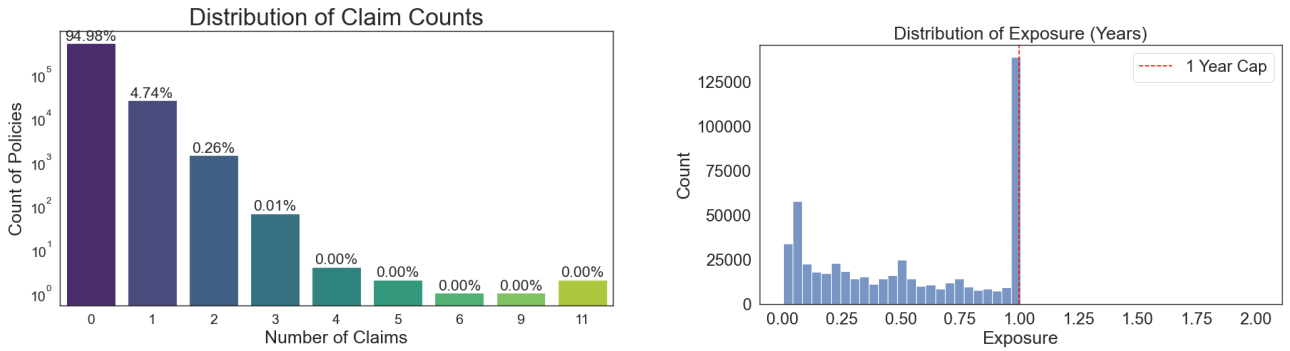
We will work with an auto insurance dataset containing an overview of vehicle insurance policies and their claim history over the past year. Our target variable is based on **ClaimNb**, the number of claims per policy, and **Exposure**, the time at risk. The dataset also includes variables such as vehicle attributes(**VehBrand, VehGas, VehPower, VehAge**), driver attributes(**VehAge**), and geographic attributes (**Area, Density, Region**), as well as insurance-specific variables such as **BonusMalus** and a policy identifier (**IDPol**).

2.2 Definition of target variables

Our target variable is defined as $E[ClaimRate] = \frac{E[Claim]}{Exposure}$, chosen because it measures claim frequency per unit time. Policies can have different lengths of time, so using just ClaimNb would unfairly favour short exposure policies, since they had less time to make claims. Dividing by Exposure accounts for how long the policy was active, allowing policies to be compared fairly.

The exploratory analysis of ClaimNb shows a strong right-skewed distribution. As shown in Figure 1, the data is highly zero-inflated: 94.98% of policies have zero claims, and 4.74% have one claim. The number of policies decreases as the claim count increases, creating a long tail to the right. Therefore, normal regression assumptions are not suitable. In Figure 2, the distribution of the Exposure variable can be seen. Most policies have an exposure of 1.0, but there are 994 policies with exposure values greater than 1. Since the dataset covers a single year, values above 1.0 are not consistent with the study period and likely reflect issues. To keep Exposure aligned with the one year, we cap Exposure at 1.0 so it represents at most one year of time at risk.

[1]Kagan. Investopedia. Understanding Underwriting Risk in Insurance and Securities



(a) Distribution of ClaimNb (log-scale)

(b) Distribution of policy Exposure

Figure 1: ClaimNB and Exposure distributions.

2.3 Exploratory Data Analysis

The dataset contains a combination of numerical and categorical features, leading to different preprocessing methods. We started by doing exploratory data analysis of the variables. The distributions of the variables were examined, as well as their correlation matrix. The findings were as follows:

In Figure 2, several numeric features are clearly right-skewed (VehAge, BonusMalus, Density, and VehPower), meaning a smaller group forms a long tail of extreme values. This is especially true for Density and BonusMalus, where a few very large values stand out.

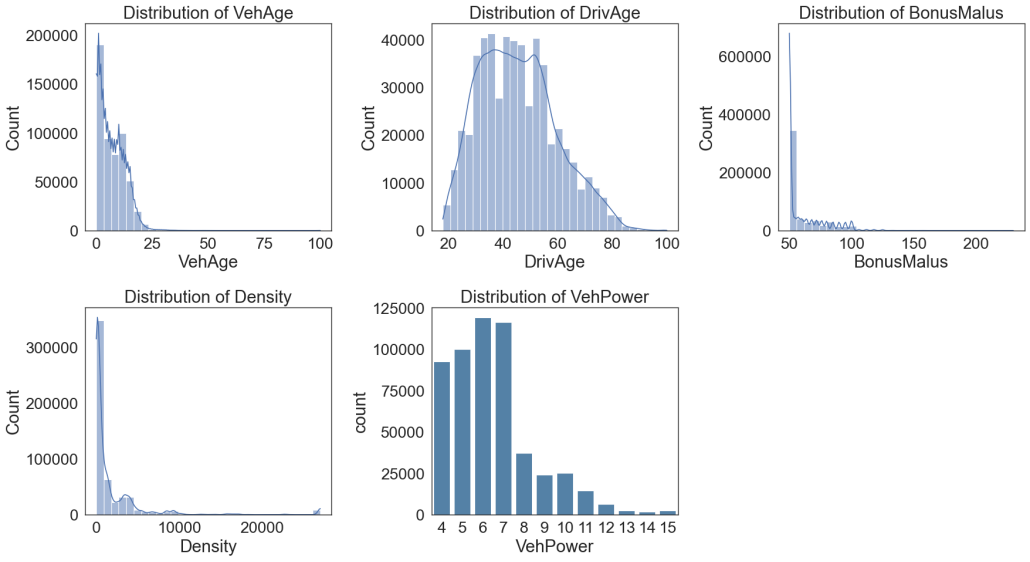


Figure 2: Distributions of numeric features

The categorical variables in Figure 3 are also imbalanced. Area is dominated by C and E, VehBrand and Region have a few very frequent categories and many rare ones, while VehGas is relatively balanced.

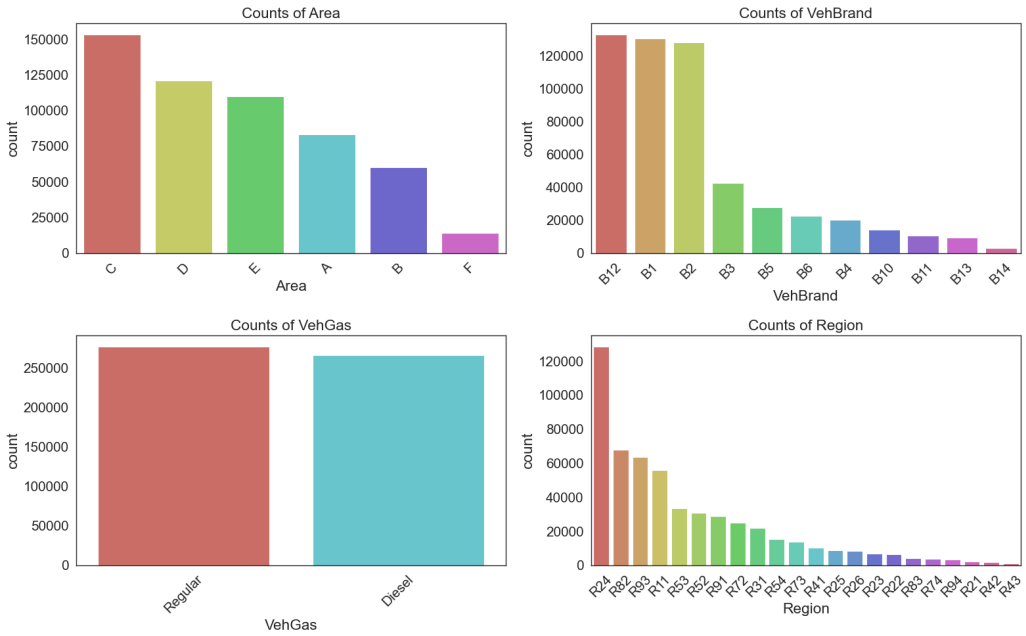


Figure 3: Distribution of categorical features

Finally, as seen in Figure 4, most feature correlations are small, but Density and Area overlap strongly, having almost perfect correlation. DrivAge is moderately negatively related to BonusMalus, suggesting that older drivers tend to have lower BonusMalus scores. Based on these EDA findings, we chose our Preprocessing methods.

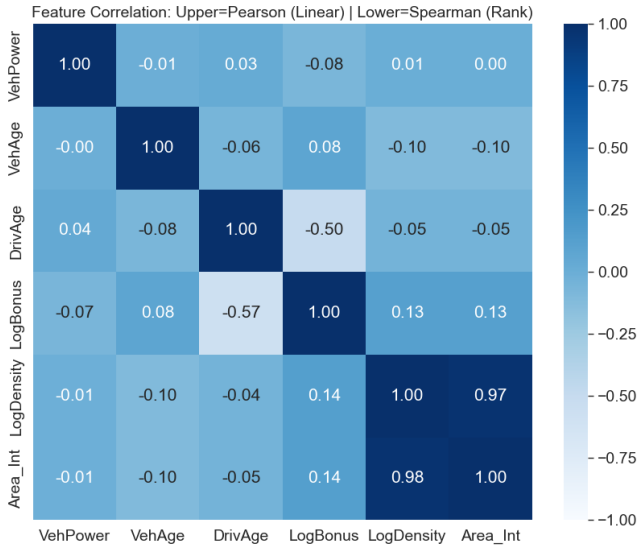


Figure 4: Feature correlations split by Person(upper) and Spearman (lower) correlations

2.4 Data Cleaning and Preprocessing

Firstly, IDpol is removed since it doesn't contain any meaningful information about claim risk. Despite the results in the correlation matrix, we decided to keep both Area and Density because Area provides categories while LogDensity shows the precise continuous variations within these categories.

For our tree-based models, the preprocessing is kept minimal because tree models are scale

invariant[2]. The variable Area has a natural order from A to F (from rural to urban), so we encode it as numbers 1 to 6 to keep that order. The remaining categorical variables are one-hot encoded, while all other numeric features are left in their original units.

For the Neural Network, we apply transformations to make gradient descent more stable and reduce the influence of the heavy-tailed predictors found during EDA. Since Density and BonusMalus are strongly right-skewed, they are log-transformed and then standardized. Finally, categorical variables are one-hot encoded, including Area. This choice was made for the variable Area since we wanted to prevent possible incorrect assumptions about linear relationships or fixed distances, allowing the model to instead learn a unique, independent weight for each region.

3 PCA and Clustering

As part of our exploratory data analysis, we applied Principal Component Analysis (PCA) to project the high-dimensional feature space into a smaller number of components that are easier to visualize and interpret. PCA creates new variables as linear combinations of the original features, ordered such that the first components explain the largest amount of variance in the data. Before applying PCA, we log-transformed the skewed variables Density and BonusMalus, one-hot encoded the categorical features, and applied scaling to all features, since PCA is based on variance.

After PCA, we applied k-means clustering on the PCA scores to observe whether the data could be grouped in the reduced feature space. The number of principal components was selected using a scree plot(Figure 5) and to keep it visualizable. The number of clusters was tuned using the silhouette score. Using three principal components the explained variance was 13.6%, and k-means with 4 clusters achieved a silhouette score of 0.346.

The scatterplots(Figure 6) shows the results. We observe in the first scatterplot that PC1 separates cluster 0 and 1 fairly well, leading us to believe that brand B12, Region R24, and vehicle age helps split these two clusters since PC1 is loaded by these features. PC2, dominated by BonusMalus and driver age, separates cluster 0 and 1 with cluster 2, and PC3, loaded by vehicle power, separates cluster 3 from the rest.

Although the clusters show some separation, there is still some overlapping. This is expected, due to the low explained variance and the data containing noise.

Nevertheless, PCA provides insight into which observations lie close to each other in the reduced space, and the PCA loadings indicate which features contribute most to the directions that spread the data and thereby influence the clustering.

[2]Gerón, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 2022, p.197

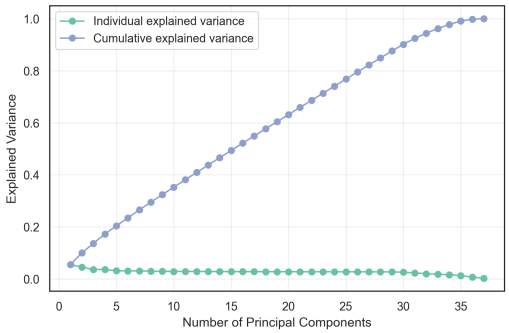


Figure 5: Scree plot showing individual and cumulative explained variance for PCA.

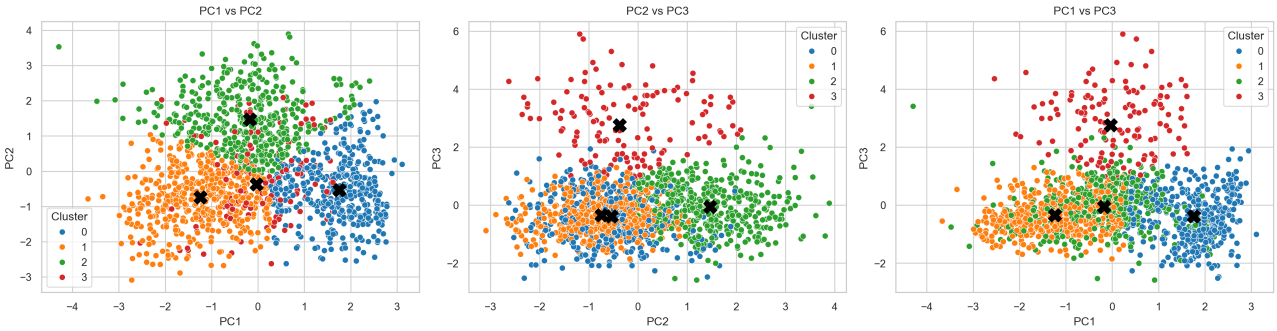


Figure 6: Two-dimensional scatter plots illustrating clustering results after PCA.

4 Details on implementations

Given that our objective is to predict claim rate, the target variable is discrete and non-negative. As Figure 1 shows, ClaimNB does not follow a Normal distribution. This makes standard metrics like Mean Squared Error (MSE) less suitable, as MSE assumes constant variance. In contrast, the variance of a Poisson distribution is equal to its mean, meaning that higher-risk drivers should have higher variance in their claim counts. In addition ClaimNB is a count variable, so a count function, like Poisson Deviance is more appropriate. [3]

The Weighted Mean Poisson deviance D is defined as[4]:

$$D(y, \hat{y}) = \frac{2}{\sum w_i} \sum_{i=1}^{n} w_i \left(y_i \log \left(\frac{y_i}{\hat{y}_i} \right) - (y_i - \hat{y}_i) \right)$$

where y_i is the observed claim rate, $\hat{y}_i$ is the predicted claim rate, and w_i is the exposure weight.

4.1 M1 Decision Tree

The Decision Tree utilizes the Poisson Deviance splitting criterion. Unlike the standard Mean Squared Error (MSE), this criterion minimizes the deviance between observed claim counts and predicted rates, explicitly weighing samples by their Exposure to account for varying policy durations. Prior to modeling, the data was processed using the tree-specific pipeline described in Section 2.4.

[3] James et al.. An Introduction to Statistical Learning with Applications in Python. 2021, Ch.4.6, pp.169-170.
[4] Jørgensen. Generalized linear models. 2013. p. 7

To optimize the model's generalization performance, we performed hyperparameter tuning using 5-fold cross-validation. We focused on balancing the bias-variance trade-off by tuning Maximum Tree Depth and Minimum Leaf Weight. We tested depths ranging from 6 to 10 to control model complexity. A deeper tree captures more patterns but risks overfitting, while a shallower tree may fail to capture the underlying risk structure[5]. Simultaneously, we tested Minimum Leaf Weight values of 10, 20, 30, 50, 100, and 250. In this context, weight refers to accumulated exposure. Requiring a minimum amount of exposure per leaf ensures that the model does not form rules based on statistically insignificant groups. The optimal structure is a maximum depth of 7 and a minimum Leaf Weight of 30 exposure. Following the initial training, we applied Cost Complexity Pruning (CCP) to further reduce overfitting. CCP introduces a regularization parameter, alpha, which penalizes the tree for its size. By identifying and removing branches where the reduction in loss score does not justify the additional complexity cost, this method produces a simpler, more robust tree that generalizes better to unseen data.

4.2 M2 Neural Network

For our second method, we implemented a Feed-Forward Neural Network. The model architecture consists of fully connected linear layers followed by ReLU activation functions to capture non-linear interactions. The final output layer uses an exponential activation function. This ensures that the predicted claim rates are strictly positive, which is a requirement for Poisson regression.

The model was trained using the AdamW optimizer to handle the sparse gradients from our one-hot encoded features. Hyperparameter tuning was performed in two stages. First, we tested different network architectures, varying depth and width, to find a structure with enough capacity to learn the risk patterns. The optimal architecture was 128 for first hidden layer and 64 for the second. Second, we tuned the learning rate and weight decay to optimize convergence and prevent overfitting. We implemented early stopping with a patience of 15 epochs. This monitored the validation loss during training and stopped the process if performance on unseen data stopped improving, ensuring the model captured the actual signal rather than memorizing noise.

4.3 M3 LightGBM

For our third method, we chose a boosting model because insurance data is highly biased, with most policies having zero claims. When evaluating the results of the Decision Tree, we observed similar performance on the validation and test sets, suggesting that the models were not overfitting. Instead, this indicates that the models are too simple to capture the underlying patterns in the data, a bias problem. Therefore, we selected a model reducing bias and specifically LightGBM. This model can reduce the loss more than depth-wise tree growth because it always chooses the most optimal split point [6]. Preprocessing was applied as mentioned in section 2.4.

The model was optimized by first doing a randomized search with 3-fold cross-validation to identify the best hyperparameters. We tuned the number of leaves and minimum child samples to control tree complexity, along with subsampling ratios and **L1/L2 regularization** to prevent overfitting.

After identifying the optimal parameters, a 5-fold ensemble training method was implemented using slow learning. We significantly reduced the learning rate to 0.005 and increased the max-

[5]GeeksforGeeks, decision-tree

[6]GeeksforGeeks, lightgbm-leaf-wise-tree-growth-strategy

imum boosting rounds to 10,000. Lowering the learning rate ensures that each new tree makes only small corrections to the overall prediction, which results in a more stable and accurate final model. During this long training process, we used early stopping, which automatically stopped the training if the validation deviance did not improve for 100 consecutive rounds. The final prediction is calculated as the average of the five models trained on the different cross-validation folds, which reduces variance and ensures the results are not biased by a specific train-validation split.

5 Results and Discussion

Table 1: Comparative Performance of Models on Test Set

Metric	**Decision Tree (M1)**	**Neural Network (M2)**	**LightGBM (M3)**
WMPD	0.58506	0.58607	0.57073
D^2 Score (%)	7.58%	7.42%	9.84%
AUC (Lorenz)	0.344	0.340	0.319
Gini Coefficient	0.313	0.320	0.362

Note: Results for Scratch implementation for M1 and M2 but library implementation for M3. WMPD is Weighted Mean Poisson Deviance. D^2 represents the percentage of deviance explained relative to the Null Model.

Table 2: Reference Models Comparative Performance of Models on Test Set

Metric	**Decision Tree (M1)**	**Neural Network (M2)**
WMPD	0.58483	0.58458
D^2 Score (%)	7.62%	7.65%
AUC (Lorenz)	0.343	0.338
Gini Coefficient	0.314	0.324

Note: Results are from reference models. Results are similar indicating no errors in scratch models.

5.1 Evaluation Metrics

To evaluate the predictive performance of our models, we used the two metrics: the D^2 score and the Gini Coefficient.

WMPD is difficult to interpret, therefore we calculated the D^2 score. This is the Poisson regression equivalent of the (R^2) used in linear regression. It measures the percentage of deviance explained by the model relative to a Null Model, which is a base model that predicts the average claim rate[7]. The formula is:

$$D^2 = 1 - \frac{D_{model}}{D_{null}}$$

$$D_{null} = \frac{2}{\sum w_i} \sum_{i=1}^{n} w_i \left(y_i \log \left(\frac{y_i}{\bar{\lambda}} \right) - \left(y_i - \bar{\lambda} \right) \right)$$

[7]Bookdown, Chapter 5.5

A higher D^2 indicates a better fit. If $D^2 = 0$, the model performs no better than a simple average; if $D^2 = 1$, the model perfectly predicts every claim.

The Gini Coefficient measures its ability to rank policies from lowest to highest risk. This is derived from the Lorenz Curve, which plots cumulative exposure against cumulative claims. The Area Under this Curve (AUC) represents the probability that a randomly chosen high claim rate policy is ranked higher than a lower claim rate policy. The Gini coefficient normalizes this value to a range of [0, 1]:

$$Gini = 2 \times AUC - 1$$

[8]

A Gini of 0 implies random ranking, while a value closer to 1 indicates that the model has successfully concentrated the riskiest drivers at the top of the distribution. This metric is scale-invariant, meaning it focuses purely on the ordering of predictions rather than their absolute magnitude.

The Gini coefficient tells us how good the model is at sorting drivers from lowest to highest risk. A Gini of 0 means the model is guessing at random. A value closer to 1 means the model successfully put the people who actually had claims at the top of the list. It is a way to see if our model can actually distinguish a high-risk policy from a low-risk policy.

5.2 Comparative Performance

Table 1 summarizes the performance of our three models on the test set. The results demonstrate that LightGBM is the superior model, outperforming both the Decision Tree and Neural Network across all evaluated metrics. LightGBM achieved the lowest WMPD (0.5707) and the highest D^2 score (9.84%), indicating it captured the most information from noisy data. The performance gap between the scratch implementations of Decision Tree and Neural Network is marginal. Although the Decision Tree achieved a slightly better D^2 of 7.58% vs 7.42%, the Neural Network showed slightly better Gini of 0.320 vs 0.313. This suggests that while the tree was better at minimizing the overall error on average, the Neural Network was slightly more effective at ranking high-risk drivers relative to low-risk ones. In general, we can not expect good performance for models when predicting claim risk. [9]

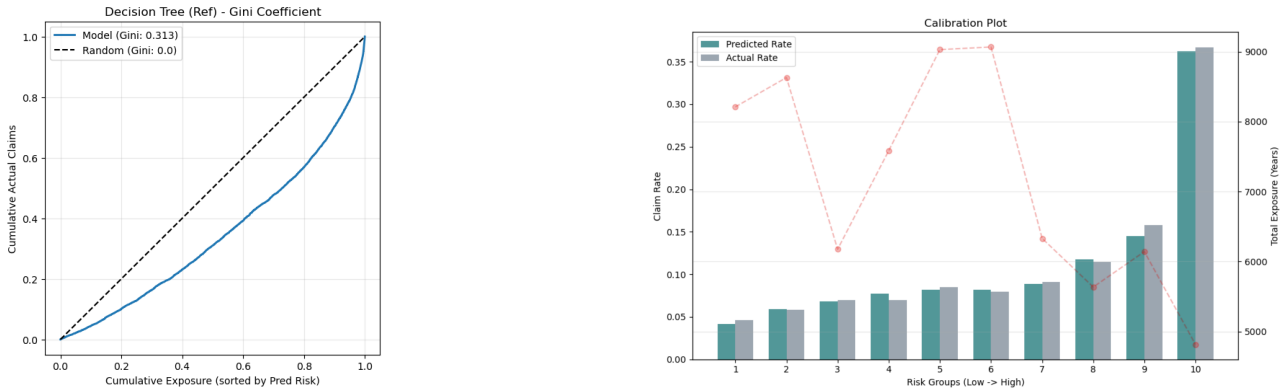
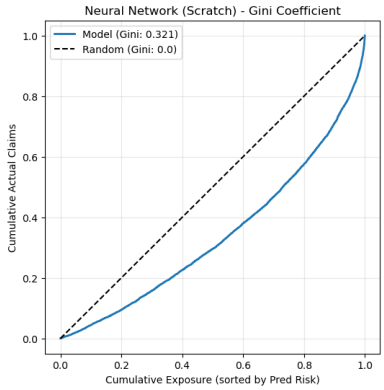


Figure 7: Decision Tree results: Lorenz Curve / Gini Coefficient and Calibration Plot

[8] Scikit-learn.org, ROC AUC score
[9] ITU. ML Project Proposal. 2025. p.4



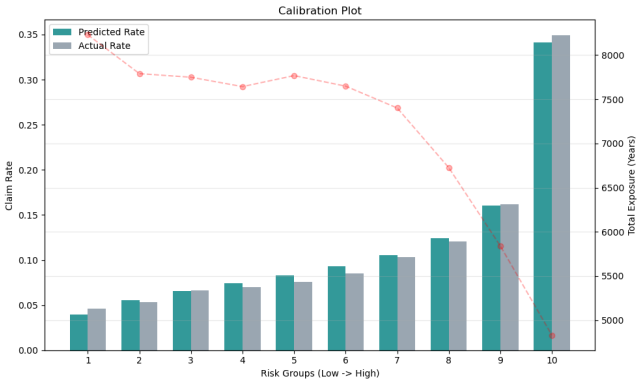
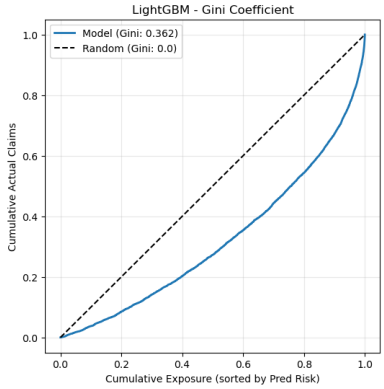


Figure 8: Neural Network results: Lorenz Curve / Gini Coefficient and Calibration Plot



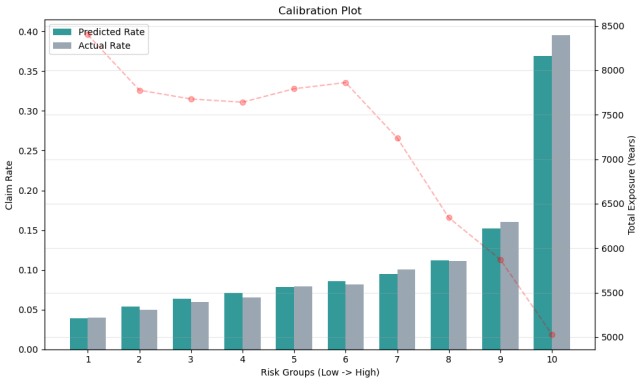


Figure 9: LightGBM results: Lorenz Curve / Gini Coefficient and Calibration Plot

6 Conclusion

In this project, we compared three models for predicting automobile insurance claim risk: a Decision Tree, a feed-forward Neural Network, and a gradient boosting model (LightGBM). The goal was to evaluate how different modeling approaches perform on highly skewed insurance data.

LightGBM clearly outperformed the other models across all metrics, achieving the lowest Poisson deviance at 0.57073 and the highest D^2 at 9.86% and a Gini coefficient score of 0.362. Claim frequency prediction is a high-bias problem, and boosting improves performance by correcting systematic errors made by simpler models.

The Decision Tree and Neural Network showed similar performance, suggesting that single models struggle to capture the full complexity of insurance risk. The similar results of our scratch implementations and reference models confirm that performance differences are due to model choice rather than implementation errors. Overall, these results suggest that boosting-based models such as LightGBM work better for modeling complex and noisy insurance data.

7 References

Bookdown, Notes for Predictive Modeling, Chapter 5.5, 2025

https://bookdown.org/egarpor/PM-UC3M/glm-deviance.html

GeeksforGeeks. (2023, October 18). LightGBM leaf-wise tree growth strategy.

Url: https://www.geeksforgeeks.org/machine-learning/lightgbm-leaf-wise-tree-growth-strategy/

GeeksforGeeks. (2025, June 30). Decision tree.

Url: https://www.geeksforgeeks.org/machine-learning/decision-tree/

Gerón. Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow. 2022

Investopedia. (2025, November 19). Understanding Underwriting Risk in Insurance and Securities

Url: https://www.investopedia.com/terms/u/underwriting-risk.asp

IT University of Copenhagen, ML Project Proposal, Fall 2025

James et al.. An Introduction to Statistical Learning with Applications in Python. 2021

Jørgensen, B. (2013). Generalized linear models. University of Southern Denmark.

Url: https://findresearcher.sdu.dk:8443/ws/files/55578494/Ben45.pdf

scikit-learn.org. ()

Url: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc$_a uc_s core.html$

7.1 Usage of GenAI

This section documents our use of Generative AI in this project, following the guidelines set by ITU. We used Google Gemini, Grammarly and ChatGPT to support specific parts of our coding and writing workflow

As outlined in the requirements for code generation, we used these AI tools to support the development of our code. The LLMs were used for debugging errors and for suggesting code structures to keep the project organized. Additionally, we used them for exploring libraries and functions, so we could use the arguments within the functions correctly. However, the actual implementation of the model, the custom loss definitions, and the training loop were performed by us. Finally, the LLMs were used to perform a grammar check or tiny tweaks on our text. The mathematical explanations and interpretations of the results remain our own work.